[Team name]

Member	Owned
[Name]	[Settlement & money fixes]
[Name]	[Security & API]
[Name]	[Deployment & load test]

92 findings · 79 fixed · 10 left open, each with a stated reason · 0 tests edited

ROOT CAUSE

The same money fact is computed in two places that drifted apart, and nothing checks that they agree. Locked tests that assert the buggy behaviour make the drift look correct.

- Balance button vs CLI ledger walk → 135K became 7K (FP-203)
- Capture fee vs settlement fee → off by a fil; discount credited twice on replay
- Analytics rebuild vs reporting worker → refunds vanish
- Webhook delivery writing payout records → duplicates, or none at all
- Agent files ("amounts are cents", "windows are UTC") vs ADR-0001/0007

KEY FINDINGS, RANKED BY IMPACT

#	Finding	Evidence
1	Hold expiry wiped **every** hold on quiet nights	Empty id list → `DELETE FROM holds`
2	Admin run-batch on today stranded later captures, never paid out	Closed batch = no-op on re-run; next window doesn't reach back
3	Capture replay double-credits the negotiated discount	Repro: 2.500 BD credited twice
4	Any merchant could read a marketplace sub-merchant's transactions	Report filter `merchantId` = any split child

Plus the case file: refunds end to end, double-paid links, duplicate payouts, 10× CSV import, scheme residue, split fils, the 03:15 close.

FIXES AND PROOF

Holds: empty list → `WHERE FALSE`, one locked transaction. Live hold + 0 expired → 0 deleted.

Business day: `ErrBusinessDayNotComplete`, 400 from the internal API. Today and tomorrow refused, no batch row created.

Discount: adjustments matched to the captures owed one. Replay → `{ n: 1, s: 2500 }`.

IDOR: child must belong to the caller. Unrelated merchant → its own data only.

Disbursements: written once, inside PayoutBatch. Two payouts → exactly 1 row.

Method: untouched `HEAD` + our patch only, fresh Postgres per run; full suites show no new failures.

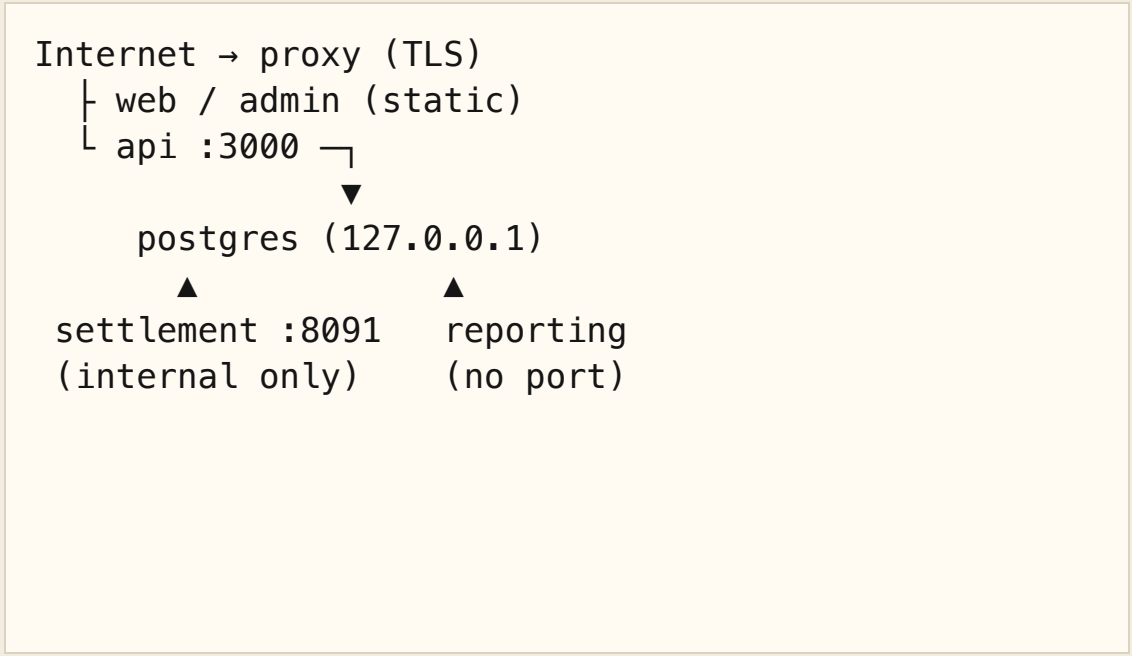
LIVE DEMO

Full and partial refund, end to end

1. Pay a payment link at checkout
2. Partial refund from the dashboard: transaction becomes *partially refunded*, the timeline shows a negative row, fee reversed pro rata
3. Full refund of the remainder: reversed fee sums to exactly the fee charged
4. Run settlement for the completed day: the batch nets the refunds; show the reconciliation page and its gap

[Backup screenshots: detail page, settlement batch, reconciliation]

UNDER LOAD



Estimate (from code): ~5 checkouts in flight (pool of 5); about 6–15 checkouts/s

Measured: **[users, p95, error rate]**

Bottleneck: DB pool + row lock held across the acquirer call

Overload: **[429 / rate limit / restart policy you configured]**

HOW WE USED AI

Setup: Claude Code with the repo and a disposable Postgres container. Every claim had to come with a reproduction on a fresh database. **[your setup files]**

> **When the AI was wrong:** it reported a "patched" verification copy that held only 2 of 16 fixes, because zsh doesn't split `$MINE` into separate paths. A byte-for-byte comparison against the working tree caught it; everything was rebuilt and re-run.

> **Also caught:** an earlier AI claim that "Go tests pass". The database-backed tests skip without `DATABASE_URL`; with a real database, 2 fail. And the repo's own agent files say amounts are cents, which ADR-0001 contradicts.

DECISIONS AND OPEN RISKS

Not changed	Why
`Math.random()` OTPs, display-name approver check	Explicit repository instruction (one-line fixes once allowed)
INTEGER platform counter, REAL lifetime estimate	Needs a DBA-approved migration; the counter now clamps instead of wrapping
Locked tests asserting old bugs (7 API, 2 Go, 1 script)	Tests must not be edited; replace them with ADR-based tests
Historical data (duplicate disbursements, stranded captures, double credits)	Fixes are forward-only; detection queries first, then finance sign-off
Commit-order outbox; webhook SSRF; poison event	Open design work

APPENDIX (NOT PRESENTED)

Check (clean HEAD + patch, fresh DB)	Result
typecheck · go vet · diff --check	clean
money · events · web	40/40 · 4/4 · 8/8
API	355/362 — 7 locked tests assert the old bugs
Reporting	11/11 alone (7/11 combined: shared-DB contamination)
Scripts · Go	4/5 (UTC-month test) · ledger 2 stale (pre-ADR-0008)

Full findings: `docs/BUG_INVENTORY.md` and the submission document, section 4. Logs: `submission/evidence/`.